

Relatório de Implementação: Simulação 3D de Cubo Mágico

Pedro Ayres Victor Dantas João Carlos Pires Roberto Campos

25 de maio de 2026

1 Objetivo do Projeto

Desenvolver uma simulação computacional interativa de um Cubo Mágico (Rubik's Cube) 3x3x3, aplicando conceitos fundamentais de Computação Gráfica, como transformações geométricas (rotação e translação), manipulação de câmera espacial e renderização em tempo real, utilizando a biblioteca **Three.js**.

2 Metodologia e Tecnologias

O projeto foi construído utilizando o ecossistema Web padrão (HTML5, CSS3, JavaScript) integrado ao motor gráfico WebGL abstraído pelo Three.js. A arquitetura do código foi dividida de forma modular:

- `index.html` e `style.css`: Estrutura da interface de usuário (UI) e HUD sobreposto ao canvas.
- `cubo.js`: Encapsulamento da classe `Cubo`, responsável pela lógica de negócio, criação das malhas geométricas e matemática das rotações.
- `main.js`: Setup do ambiente gráfico (Cena, Câmera, Luzes e Loop de Renderização).

3 Detalhamento Técnico (Implementação)

3.1 Modelagem Geométrica e Materiais

O cubo principal é composto por 27 sub-cubos menores (cubinhos).

- Utilizou-se a primitiva `THREE.BoxGeometry` para instanciar cada malha.
- O mapeamento de cores seguiu o padrão real do brinquedo, aplicando um array de `THREE.MeshBasicMaterial` para pintar as 6 faces isoladamente. As faces internas (não visíveis inicialmente) receberam cor escura para otimização visual.

3.2 Câmera e Iluminação

- **Câmera:** Utilizada a `PerspectiveCamera` para simular o campo de visão humano. Acoplada ao `OrbitControls`, permite ao usuário inspecionar o objeto em 360 graus de forma livre.
- **Luz:** Implementada uma iluminação mista, com `AmbientLight` para preenchimento geral e `DirectionalLight` para criar contraste direcional nas faces.

3.3 Transformações Geométricas (Rotação)

Para solucionar o complexo problema de rotacionar fatias do cubo em torno de um eixo comum sem desalinhar os blocos individuais, adotou-se a estratégia de agrupamento dinâmico:

1. O sistema identifica quais cubinhos pertencem à face acionada (ex: coordenada $y = 1$ para a face Superior).
2. Os blocos identificados são removidos temporariamente da cena e anexados a um pivô centralizado (`THREE.Group()`).
3. O grupo sofre uma rotação interpolada suave baseada na velocidade do frame.
4. Ao atingir o limite (90 graus ou $\pi/2$ radianos), as matrizes de mundo são atualizadas (`updateMatrixWorld`), os blocos recebem sua nova coordenada espacial absoluta e são devolvidos à cena principal.

4 Funcionalidades Interativas

A simulação não é apenas estática; possui um ciclo de interação completo (*Loop Event*), acessível via interface de botões ou mapeamento de teclado:

- **Movimentos Independentes:** Rotação das faces Superior (U), Frontal (F) e Inferior (D).
- **Sistema de Shuffle (Embaralhamento):** Automação que sorteia movimentos aleatórios aplicando rotações sequenciais temporizadas (`setInterval`).
- **Controle de Estado:** Botão para resetar (recriar) a matriz do cubo e contador dinâmico de movimentos realizados na sessão.
- **Zoom de Câmera:** Mapeamento de escalas vetoriais através das teclas + e -.

5 Conclusão

O desenvolvimento atendeu aos requisitos técnicos esperados. O uso prático do Three.js permitiu consolidar na prática os conceitos matemáticos de manipulação de matrizes no espaço 3D, estrutura de cena (*Scene Graph*) e gestão de estados de animação vinculados à taxa de quadros (*requestAnimationFrame*).